

MLGWorks.Utils

Practical Unity utilities and patterns

TrickShotMLG02

August 12, 2026

Abstract

MLGWorks.Utils is a small Unity assembly for reusable infrastructure: serializable collections, pooling, timing, scene loading, dependency injection, logging, and everyday gameplay patterns. This document follows the source tree and focuses on how the current implementation is meant to be used.

Contents

1	Getting Started	3
2	Timer	4
2.1	Lifecycle	4
3	Collections	5
3.1	Runtime collections	5
3.2	Editor support	5
4	Pooling	7
4.1	Core pooling	7
4.2	Unity pooling	7
5	Timing helpers	9
5.1	Cooldown	9
5.2	RateLimiter	9
6	Scene management	10
6.1	Scene references	10
6.2	Loading and coordination	10
6.3	Transitions	10
7	Unity extensions	11
7.1	Random collections	11
7.2	Components and transforms	11
7.3	UI and Unity object helpers	11
8	Dependency injection	12
8.1	Registering and resolving services	12
8.2	Injection	12
8.3	Startup and auto-registration	12

9	Logging	13
9.1	Setup	13
9.2	File routing	13
9.3	Lifecycle	13
10	Composite disposables	14
11	Event bus	15
12	Singletons	16
12.1	Unity singleton	16
12.2	Pure C# singleton	16
13	State machine	17
13.1	States and transitions	17
13.2	Update behavior	17

1 Getting Started

The code is delivered as an embedded Unity assembly through `MLGWorks.Utills.asmdef`. Add the repository below `Assets/MLGWorks.Utills` in a consuming project, or copy the folder into an existing Unity project. The examples assume Unity 6 and the namespaces shown in each section.

Most types are ordinary C# classes and can be constructed and tested without a MonoBehaviour. Unity-facing helpers are kept in the relevant `Unity` or `Editor` folders.

```
using MLGWorks.Utills.Helpers.Unity;  
  
var values = new[] { "red", "green", "blue" };  
var shuffled = values.Shuffled();
```

2 Timer

`MLGWorks.Utils.Helpers.Timer` is a plain C# countdown timer. It owns no Unity object and advances only when the caller supplies a delta time. That makes it useful in a `MonoBehaviour`, a service, or a deterministic test.

2.1 Lifecycle

- Start()** Resets elapsed time and starts the timer.
- Pause()** Freezes a running timer without resetting it.
- Resume()** Continues a paused timer.
- Reset()** Stops the timer and clears elapsed time.
- Update(float)** Advances the timer and fires `OnFinished` once at the duration.

Useful state is exposed through `Duration`, `Elapsed`, `IsRunning`, `IsPaused`, and `IsFinished`.

```
using MLGWorks.Utils.Helpers;
using UnityEngine;

public sealed class RespawnTimer : MonoBehaviour
{
    private readonly Timer timer = new Timer(3f);

    private void Awake()
    {
        timer.OnFinished += Respawn;
        timer.Start();
    }

    private void Update() => timer.Update(Time.deltaTime);

    private void Respawn() => Debug.Log("Respawn is ready.");
}
```

`Start()` always starts from zero. When the timer reaches its duration, it clamps `Elapsed`, stops running, clears the paused state, and invokes the event.

3 Collections

Unity serializes fields, arrays, and lists well, but it does not directly serialize the standard .NET dictionary, hash set, or one-to-many lookup types. The collection helpers under **Helpers/Collections** keep a serialized representation for the Inspector and rebuild a runtime representation for fast access.

3.1 Runtime collections

SerializableDictionary `SerializableDictionary<TKey,TValue>` provides dictionary operations such as `Add`, `TryAdd`, `TryGetValue`, `Remove`, `AddRange`, and `RemoveWhere`. Serialized entries retain Inspector order. Duplicate keys are handled through `DuplicateKeyPolicy`: keep the first value, keep the last value, or report the duplicate without silently choosing one.

SerializableHashSet `SerializableHashSet<T>` stores unique values and provides `Contains`, `Add`, `Remove`, bulk updates, and predicate-based removal. Serialized order is retained for Inspector display.

SerializableLookup `SerializableLookup<TKey,TValue>` maps one key to multiple values. Use `Add`, `GetValues`, or `TryGetValues` when a dictionary value would otherwise need to be a manually managed list.

All three types derive from `SerializableCollectionBase`. They expose validation results, change/rebuild events, read-only interfaces, and explicit `Rebuild()` behavior. Null entries, malformed entries, and duplicate keys are represented as validation issues with a severity and an explanatory message.

```
using System;
using MLGWorks.Utills.Helpers.Collections;
using UnityEngine;

[Serializable]
public sealed class ItemTags : SerializableLookup<string, string> { }

public sealed class InventoryData : MonoBehaviour
{
    [SerializeField] private ItemTags tags = new ItemTags();

    private void Reset()
    {
        tags.Clear();
        tags.Add("Sword", "Weapon");
        tags.Add("Sword", "Melee");
        tags.Add("Potion", "Consumable");
    }
}
```

The repository also contains a matching Inspector example. The **Editor** side adds the default property drawer, extension registration, and editor-only copy, paste, duplicate, and in-memory preset operations.

3.2 Editor support

The collection editor code is not part of the runtime API. The default property drawer draws supported collection fields in the Inspector, while the drawer registry discovers and selects custom extensions.

Custom editor drawers implement `ICollectionDrawerExtension` and are associated with a collection type through an attribute. The editor clipboard provides commands for copying, pasting, duplicating, and saving or loading a current in-memory preset.

These tools affect the Unity Editor only; they do not add editor dependencies to player builds.

4 Pooling

Pooling is useful when a game repeatedly creates and releases short-lived objects, such as projectiles, enemies, audio sources, or particle effects. The implementation is split between a general-purpose core and Unity wrappers.

4.1 Core pooling

`ObjectPool<T>` owns a factory, inactive objects, checked-out objects, and optional lifecycle callbacks. It is not thread-safe and does not impose a reset convention, although objects implementing `IPoolable` automatically receive `OnPoolAcquire()` and `OnPoolRelease()` calls.

```
using MLGWorks.Utils.Helpers.Pooling.Core;

var pool = new ObjectPool<List<int>>>(
    factory: () => new List<int>(),
    onRelease: list => list.Clear(),
    initialCapacity: 4,
    maxCapacity: 16);

List<int> values = pool.Get();
values.Add(42);
pool.Release(values);
pool.Dispose();
```

The main operations are:

- `Get()` and `GetMany(...)` acquire objects and mark them active.
- `Release(...)` and `ReleaseMany(...)` return active objects or destroy them when the retention limit is reached.
- `Prewarm(...)` creates all requested inactive objects immediately.
- `PrewarmStep(...)` creates up to a requested number, making it possible to spread a large warm-up across frames.
- `Clear()` destroys inactive objects; `Dispose()` destroys everything and prevents further use.

`CountInactive`, `CountActive`, and `CountAll` expose pool size. A null factory result, use after disposal, invalid release, or invalid capacity is reported with an exception.

4.2 Unity pooling

The Unity wrappers use prefab instances while keeping the same acquire/release model as `ObjectPool<T>`:

- `GameObjectPool` activates acquired prefab instances and deactivates released ones.
- `ComponentPool<T>` acquires components from pooled `GameObjects` and supports `IPoolable` hooks.
- `AudioSourcePool` resets playback state and can acquire a source with an optional clip and immediate playback.
- `ParticleSystemPool` stops and resets particle systems between uses.

```
using MLGWorks.Utils.Helpers.Pooling.Unity;
using UnityEngine;

public sealed class EffectSpawner : MonoBehaviour
{
    [SerializeField] private GameObject effectPrefab;
    private ParticleSystemPool pool;

    private void Awake() => pool = new ParticleSystemPool(effectPrefab);

    public ParticleSystem Spawn(Vector3 position)
    {
        ParticleSystem effect = pool.Get();
        effect.transform.position = position;
        effect.Play();
        return effect;
    }

    public void Despawn(ParticleSystem effect) => pool.Release(effect);
    private void OnDestroy() => pool?.Dispose();
}
```

All Unity pools provide prewarming, stepped prewarming, clearing, and disposal. They own the instances they create, so release instances through the same pool that acquired them.

5 Timing helpers

The stateful timing helpers under `Helpers/Timing` are ordinary C# objects. The caller supplies time, which keeps them useful in both Unity update loops and tests.

5.1 Cooldown

`Cooldown` exposes `Duration`, `Remaining`, `CanConsume`, and `IsActive`. Call `TryConsume()` to start the cooldown only when it is ready, then call `Update(deltaTime)` each frame.

```
var attackCooldown = new Cooldown(0.5f);

if (attackCooldown.TryConsume())
{
    Attack();
}

attackCooldown.Update(Time.deltaTime);
```

`Start()` restarts the configured duration and `Reset()` returns the cooldown to its ready state.

5.2 RateLimiter

`RateLimiter` limits acquisitions in a rolling window. Construct it with a maximum number of uses and a window length, then pass a monotonic timestamp to `TryAcquire(double)`.

```
var limiter = new RateLimiter(maxUses: 3, windowSeconds: 1.0);
double now = Time.realtimeSinceStartupAsDouble;

if (limiter.TryAcquire(now))
{
    SendRequest();
}
```

Timestamps must be finite and must not move backwards. `Reset()` clears the recorded acquisition times.

6 Scene management

Scene management is split into references, loading operations, coordination, and transitions. The API validates scenes before asking Unity to load them and keeps asynchronous work in disposable operation wrappers.

6.1 Scene references

SceneReference is a serializable **ScriptableObject** reference to a scene asset. It exposes the scene path, name, display name, notes, validity, and whether the scene is present in Build Settings.

Create scene-reference assets through the Unity Editor, assign them to fields, and pass them to **SceneLoader**. Keeping the reference in an asset avoids scattering scene path strings through gameplay code.

6.2 Loading and coordination

SceneLoader provides validated synchronous and asynchronous load and unload methods. It also exposes **CanLoad**, **IsLoaded**, and **GetLoadedScene**. Asynchronous loading returns **SceneLoadOperation**; unloading returns **SceneUnloadOperation**.

```
using MLGWorks.Utils.Helpers.SceneManagement.Core;
using MLGWorks.Utils.Helpers.SceneManagement.Loading;
using UnityEngine.SceneManagement;

public sealed class GameFlow
{
    public void LoadLevel(SceneReference level)
    {
        SceneLoader.LoadAsync(
            level,
            LoadSceneMode.Single,
            setActiveScene: true,
            onCompleted: scene => UnityEngine.Debug.Log(scene.name),
            onFailed: error => UnityEngine.Debug.LogException(error));
    }
}
```

For a gameplay flow that may request the same scene more than once, use a load coordinator. It tracks active loads and reports progress when its owner calls **Tick()**. Duplicate requests either return the existing operation or throw, according to the configured duplicate-load policy. Dispose the coordinator when its owner is done.

6.3 Transitions

ISceneTransition separates visual transition work from scene loading. Its outgoing transition calls a callback when loading may begin; its incoming transition calls a callback when the transition is complete.

SceneLoader.LoadWithTransition(...) returns a **SceneTransitionOperation** that combines the outgoing transition, the scene load, and the incoming transition. The operation exposes progress, completion, failure, **Cancel()**, and **Dispose()**.

This keeps a fade, loading screen, or wipe effect reusable without coupling it to a particular scene or loading coordinator.

7 Unity extensions

The extension methods in `Helpers/Unity` focus on small operations that are common in Unity scripts.

7.1 Random collections

`RandomCollectionExtensions` provides Fisher–Yates shuffling and weighted selection. `Shuffle()` changes an `ICollection<T>` in place; `Shuffled()` creates a new `List<T>` without changing its source. Overloads accepting `System.Random` are useful for deterministic tests or seeded gameplay.

```
using System;
using System.Collections.Generic;
using MLGWorks.Utls.Helpers.Unity;

var cards = new List<string> { "A", "K", "Q", "J" };
cards.Shuffle(new Random(42));
List<string> copy = cards.Shuffled();

var loot = new[] { "Potion", "Sword", "Gem" };
var weights = new[] { 70f, 25f, 5f };
string drop = loot.SelectWeighted(weights);
```

Weights must be finite and non-negative, and the item and weight lists must have the same length. `TrySelectWeighted` returns `false` for an empty list or a zero total weight; `SelectWeighted` throws in those cases.

7.2 Components and transforms

`ComponentExtensions` provides component lookup on a `GameObject` and its children or parents. `TransformExtensions` includes hierarchy-wide layer and active-state changes, local-transform reset, child destruction, and path lookup.

```
if (gameObject.TryGetComponentInChildren(out Rigidbody body))
{
    body.gameObject.SetActiveHierarchy(false);
}

transform.ResetLocalTransform();
transform.DestroyChildren();
```

7.3 UI and Unity object helpers

`UIExtensions` avoids redundant UI assignments for anchored position, size delta, and Canvas-Group visibility. `UnityObjectExtensions` provides change-aware `GameObject` operations, safe parenting, and `SafeDestroy`, which uses the appropriate destruction method in play mode and edit mode.

8 Dependency injection

The dependency-injection utilities provide a small service workflow for Unity projects. They do not require a container package: services are registered with `ServiceLocator`, fields or properties are marked with `[Inject]`, and `Injector` supplies matching services.

8.1 Registering and resolving services

`ServiceLocator` registers one object per type and exposes `TryGet`, `GetAllServices`, and `Clear`. Registration is explicit, so startup order remains visible in the scene.

```
using MLGWorks.Utills.DependencyInjection;

ServiceLocator.Register(typeof(GameSettings), new GameSettings());

if (ServiceLocator.TryGet(typeof(GameSettings), out object value))
{
    GameSettings settings = (GameSettings)value;
}
```

8.2 Injection

Mark a field or property with `[Inject]` and call `Injector.Inject(target)`. `DIInitializer` performs this work for scene objects, including inactive objects, and offers `InitializeNow()` when a caller needs to trigger the pass explicitly.

```
public sealed class HudController : MonoBehaviour, IInitializable
{
    [Inject] private GameSettings settings;

    public void Initialize()
    {
        ApplyTheme(settings.Theme);
    }
}
```

8.3 Startup and auto-registration

`ServiceBootstrapper` is intended for registering core services early in the Unity lifecycle. A `MonoBehaviour` marked with `[DIService]` can be discovered and registered by the initializer. Implement `IInitializable` when a service needs a callback after injection is complete.

The attribute types live under `Dependency Injection/Attributes`; the runtime types remain in the parent namespace `MLGWorks.Utills.DependencyInjection`.

9 Logging

`MLGWorks.Utills.Logging.Logger` is a Unity singleton that mirrors application messages to the Unity console and writes configured log files. It queues entries, so Unity's threaded log callback can translate messages without performing file work on the callback thread.

9.1 Setup

Add a `GameObject` with the `Logger` component to a scene. Configure `pathType`, `relativePath` or `customPath`, `fileExtension`, `maxLogFileCount`, and whether debug messages should be enabled. File writers are initialized lazily on the first application message.

```
using MLGWorks.Utills.Logging;

Logger.Info("Application started.");
Logger.Warning("A fallback value was used.");
Logger.Error("The save operation failed.");
```

The static methods are `Debug`, `Info`, `Warning`, and `Error`. `EmitTestLogs()` emits one message at each standard level, which is useful when checking a project setup.

9.2 File routing

`LogLevel` values are routed by bit masks. Bit 0 is the debug file, bit 1 the info file, bit 2 the warning file, bit 3 the error file, and bit 4 the combined file. For example, $(1 \ll 0) \mid (1 \ll 4)$ sends debug entries to both debug and combined files.

`LogLocationType.PersistentDataPath` is the normal application location; `DataPath` uses the project data path, and `Custom` uses `customPath`. `CleanupOldLogFiles()` removes the oldest complete timestamped sessions until `maxLogFileCount` is satisfied. A negative maximum keeps all sessions.

9.3 Lifecycle

The logger flushes its queue during `Update()`, unsubscribes from Unity log callbacks, and closes writers during shutdown. In editor tests, `FlushAndShutdown()` is available to force the same cleanup path immediately. The editor integration is in `Logging/Editor`.

10 Composite disposables

`CompositeDisposable` owns several `IDisposable` values and releases them as one scope. It disposes resources in reverse addition order. Adding to an already disposed composite disposes the new resource immediately.

```
using MLGWorks.Utils.Patterns;

using var lifetime = new CompositeDisposable();
lifetime.Add(EventBus.Subscribe<PlayerDiedEvent>(OnPlayerDied));
lifetime.Add(otherResource);

// Both resources are disposed when the scope ends.
```

The type is intentionally small and is not thread-safe. Use one owner and one execution context for its lifetime.

11 Event bus

The static `EventBus` provides typed publish/subscribe communication without requiring publishers to know their subscribers. An event implements `IEvent`, which exposes a human-readable `Name`.

```
using MLGWorks.Utills.Patterns;

public sealed class PlayerDied : IEvent
{
    public string Name => "PlayerDied";
    public string PlayerName { get; }
    public PlayerDied(string playerName) => PlayerName = playerName;
}

IDisposable subscription = EventBus.Subscribe<PlayerDied>(OnPlayerDied);
EventBus.Publish(new PlayerDied("Alice"));
subscription.Dispose();
```

`Subscribe<T>` returns an `IDisposable`; disposing it removes exactly that subscription and is safe to repeat. `Unsubscribe<T>` is also available when the original callback is convenient to retain.

During publication the bus iterates over a snapshot, so a callback can dispose its own subscription or add another subscription without invalidating the current delivery. The bus itself is not thread-safe; use it from the Unity main thread.

12 Singletons

The singleton helpers now live in `MLGWorks.Utills.Patterns.Singletons`. They share small internal state-management code but intentionally have different creation and lifetime rules.

12.1 Unity singleton

`Singleton<T>` is a `MonoBehaviour` base for a component that already exists in a scene. During `Awake()`, the first instance becomes the owner; duplicate components are destroyed. `Instance` searches the scene if necessary and throws `InvalidOperationException` when no component exists. `OnDestroy()` releases the ownership.

```
using MLGWorks.Utills.Patterns.Singletons;

public sealed class AudioManager : Singleton<AudioManager>
{
    protected override void Awake()
    {
        base.Awake();
        // Additional initialization.
    }
}

AudioManager.Instance.PlayMusic();
```

This base does not create a `GameObject`, call `DontDestroyOnLoad`, or persist the object across scenes. Derived overrides should call the base `Awake()` and `OnDestroy()` methods.

12.2 Pure C# singleton

`PureSingleton<T>` is for regular C# classes. The instance is created lazily on first access and requires a public parameterless constructor.

```
using MLGWorks.Utills.Patterns.Singletons;

public sealed class SaveService : PureSingleton<SaveService>
{
    public void Save() { /* write save data */ }
}

SaveService.Instance.Save();
```

The pure variant has no Unity lifecycle and shares its instance for the lifetime of the application domain. Prefer dependency injection when explicit ownership or easy replacement in tests matters more than global access.

13 State machine

The state-machine types under `Patterns/StateMachine` provide a small, testable finite-state machine. A state implements `IState`; a transition connects a source to a target; and an optional `ITransitionCondition` decides whether a conditional transition is ready.

13.1 States and transitions

States receive `OnEnter()`, `OnExit()`, and `Tick()` callbacks. `UnconditionalTransition` always fires when its source is current. `ConditionalTransition` delegates its decision to an `ITransitionCondition`. Adding a transition associates the states and condition with the owning state machine through `SetFSM`.

```
using MLGWorks.Utills.Patterns.StateMachine;
using MLGWorks.Utills.Patterns.StateMachine.Interfaces;

sealed class IdleState : IState
{
    public void OnEnter() { }
    public void OnExit() { }
    public void Tick() { }
    public void SetFSM(StateMachine fsm) { }
}

var machine = new StateMachine();
var idle = new IdleState();
machine.SetStartingState(idle);
machine.AddTransition(idle, new RunningState());
machine.Tick();
```

13.2 Update behavior

Call `SetStartingState` before the first update. `ChangeState` immediately exits the old state and enters the new state. On `Tick()`, the machine evaluates registered transitions in order; if the first matching transition is ready, it changes state and returns. Otherwise it ticks the current state.

`AddTransition(IState, IState)` is shorthand for an unconditional transition. Use `AddTransition(ITransition)` when a conditional transition or custom transition is needed. `ClearTransitions()` removes all registered transitions.